

Python I/O file:

There are two types of files that can be handled in Python, normal text files and binary files (written in binary language, 0s, and 1s). Opening a file refers to getting the file ready either for reading or for writing. This can be done using the `open ()` function. This function returns a file object and takes two arguments, one that accepts the file name and another that accepts the mode (Access Mode).

1.Create a new file:

```
file=open("hello.txt","x")  
  
file.close()
```

2.Write to a new file:

```
file=open("hello.txt","w")  
  
file.write("Hello, World!")  
  
file.close()
```

3.Read from a file:

```
file=open("hello.txt","r")  
  
print(file.read())  
  
file.close()
```

4.Append to a file:

```
file=open("hello.txt","a")  
  
file.write("This a new text.")  
  
file.close()
```

5.Read line by line:

```
file=open("hello.txt","r")  
  
for line in file:  
    print(line.strip())  
  
file.close()
```

6.Write multiple line:

```
lines=["Apple","Mango","Cherry"]  
  
file=open("hello.txt","w")  
  
for line in lines:  
    file.write(line+"\n")  
  
file.close()
```

7.Copy file content:

```
with open("hello.txt","r") as source:
    with open("destination.txt","w") as destination:
        destination.write(source.read())
```

8.Check if the file exist:

```
import os
if os.path.isfile("hello.txt"):
    print("File exists.")
else:
    print("File doesn't exists.")
```

9.Delete a file:

```
import os
if os.path.isfile("hello.txt"):
    os.remove("hello.txt")
    print("File deleted.")
else:
    print("File doesn't exists.")
```

10.Write to csv file:

```
import csv
with open("data.csv","w",newline="") as file:
    writer=csv.writer(file)
    writer.writerow(["Name","Age"])
    writer.writerow(["Liver","18"])
```

11.Read JSON file:

```
import json
with open("data.json","r") as file:
    data=json.load(file)
    print(data)
```

Modes used in python file handling:

Mode	Description
r	It opens a file for reading. (By default)
w	It opens a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
x	It opens a file for exclusive creation. If the file already exists, the operation fails.
a	It opens a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
t	Opens in text mode. (By default)
b	Opens in binary mode
+	Opens a file for updating (reading and writing)

Operations/Functions on file in python:

1.open(): Opens a file and returns a file object.

2.read(): Read data from the file.

3.write(): Writes data to a file.

4.close(): Closes the file, releasing its resources.

5.with....open(): This open the file using with....open() and automatically closes the file, so there is no use of close() function.

GUI Programming in Python:

What is GUI: GUI stands for Graphical User Interface, and it's a visual way for users to interact with a computer program or system. In Python, a GUI is a vital part of the user experience, and it's made up of visual elements like buttons, icons, graphics, and text.

Advantages of GUI:

1. GUI is very user-friendly.
2. GUI is more attractive and multi-coloured.
3. It is much easy than the command-driven interface.
4. User can switch easily between tasks on the GUI interface.

Disadvantages of GUI:

1. It becomes complex if the user needs to communicate with the computer directly.
2. It is fully based applications require more RAM.
3. GUI uses more processing power compared to other interface types.

Introduction to GUI library:**What is GUI library:**

Tkinter is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Python provides various options for developing graphical user interfaces (GUIs) such as Tkinter, wxPython, JPython.

Tkinter: Tkinter is the Python interface to the Tk GUI toolkit shipped with Python. Tkinter is the standard GUI library for Python. Tkinter provides a easy as well as fast way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit.

wxPython : This is an open-source Python interface for wxWindows

JPython: JPython is a Python port for Java which gives Python scripts.

Layout Management: Tkinter has three built-in layout managers: the pack, grid, and place managers.

1.Pack method: The pack method of geometry manager organizes the widget in blocks before they appear on the parent widget. The pack geometry manager organizes widgets in horizontal and vertical boxes.

Syntax: widget.pack(pack option)

Ex:

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Hello, Tkinter!")

label.pack()
```

2.Grid method: Grid method geometry manager is used to organize widgets in a tablelike structure in the parent widget. The grid geometry manager places widgets in a 2D grid.

Syntax: widget.grid(options)

There are following option:

1.Column: It is a column to put widget in default 0.

2.Row: It is a row to put widget in. Default the first row that is still empty.

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Grid Example")

btn1 = tk.Button(root, text="Button 1")
btn1.grid(row=0, column=0)

btn2 = tk.Button(root, text="Button 2")
btn2.grid(row=0, column=1)

btn3 = tk.Button(root, text="Button 3")
btn3.grid(row=1, column=0)

btn4 = tk.Button(root, text="Button 4")
btn4.grid(row=1, column=1)

root.mainloop()
```

3.Place method: The place method of layout manager organizes widgets in the parent widget by placing them in a specific position. The place geometry manager positions widgets using absolute positioning.

Syntax: widget.place(place option)

Ex:

```
import tkinter as tk

root = tk.Tk()

label = tk.Label(root, text="Hello!")
label.place(x=20, y=20)

root.mainloop()
```

Fonts: The Simple Tuple fonts are commonly use for specify the font. The tuple contains first element font family, Second element size in points and third element style modifiers like underline, bold etc.

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Font Example")

label = tk.Label(root, text="Hello, Tkinter!", font=("Arial", 24))

label.pack()

root.mainloop()
```

Colours: Color is represented in the string format.

User can specify the color in the following:

Name: User can use any locally defined standard color name like “red”, “green”, “white”, “black”, “green” etc.

Hexadecimal unit: Instead of color name use hexadecimal digit like ‘#fff’ for white, ‘#00000’ for black, ‘#000fff000’ for pure green.

Background: bg is used to set background color for the active widget.

Foreground: fg is used to set foreground color for the active widget.

Ex:

```
import tkinter as tk

def change_color():
    root.configure(bg="skyblue")

root = tk.Tk()

root.title("Simple Color Change")

color_button = tk.Button(root, text="Change Color", command=change_color)

color_button.pack(pady=20)

root.mainloop()
```

Events and Bindings:

Events: Events in tkinter are all the command inputs that can happen inside your application such as mouse movements / clicks, keyboard entries, window resizes, widget introduction, the list goes on.

Bindings: The binding function is used to deal with the events. We can bind Python's Functions and methods to an event as well as we can bind these functions to any particular widget.

Syntax: widget.bind(event,handler)

event: Can be button clicked or key press etc.

handler: Can be type of button or key used to handle event.

Events and its Description:

<button>: A mouse button is pressed with the mouse pointer over the widget. If you press down a mouse button over a widget and keep it pressed.

<motion> (x,y): The mouse is moved with a mouse button being held down. The current position of the mouse pointer is provided in the x and y members of the event object passed to the callback, i.e. event.x, event.y

<button release>: Event, if a button is released.

<double button>: Similar to the Button event, see above, but the button is double clicked instead of a single click.

Ex:

```
import tkinter as tk

def change_label():
    label.config(text="You clicked the button!")

root = tk.Tk()

root.title("Simple Tkinter Example")

label = tk.Label(root, text="Hello!")

label.pack()

button = tk.Button(root, text="Click Me", command=change_label)

button.pack()

root.mainloop()
```

Canvas:

1.line: The create_line method creates a line item on the Canvas.

Ex:

```
fromtkinter import *  
  
t = Tk()  
  
C = Canvas(t, bg="green", height=350, width=400)  
  
coord = 10, 50, 240, 210  
  
arc = C.create_line(55, 85, 155, 85, 105, 180, 55, 85)  
  
C.pack()  
  
t.mainloop()
```

2.oval: The create_oval() method is used to create a circle item. The first four parameters are the bounding box coordinates of the circle. In other words, they are x and y coordinates of the top-left and bottom-right points of the box, in which the circle is drawn.

Ex:

```
fromtkinter import *  
  
t = Tk()  
  
C = Canvas(t, bg="green", height=350, width=400)  
  
coord = 10, 50, 240, 210  
  
arc = C.create_oval(10, 10, 350, 200, width=1)  
  
C.pack()  
  
t.mainloop()
```

3.rectangle: To create a rectangle create_rectangle() method is used. This method accepts 4 parameters x1, y1, x2, y2. Here x1 and y1 are the coordinates for the top left corner and x2 and y2 are the coordinates for the bottom right corner.

Ex:

```
fromtkinter import *  
  
t = Tk()  
  
C = Canvas(t, bg="green", height=350, width=400)  
  
coord = 10, 50, 240, 210  
  
arc = C.create_rectangle(50, 50, 290, 260, width=2)  
  
C.pack()
```



```
t.mainloop()
```

Widgets:

1.frame: The frame widget is used to group the widget in a friendly way. Its helps to look the GUI organized. It is like a container which arranges the position of the other widget.

Syntax: F=Frame (master, option)

Ex:

```
import tkinter as tk

root = tk.Tk()

frame = tk.Frame(root)

frame.pack()

label = tk.Label(frame, text="Hello, Tkinter!")

label.pack()

root.mainloop()
```

2.label: The label widget is a display box where we can place text or images. We can change label text any time we want.

Syntax: x=Label(master, option, ...)

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Tkinter Label Example")

label = tk.Label(root, text="Hello, Tkinter!", font=("Arial", 24))

label.pack(pady=20)

root.mainloop()
```

3.button: The button widget is used to add buttons in a Python application. These buttons can display text or images that convey the purpose of the buttons.

Syntax:**Ex:**

```
import tkinter as tk

root = tk.Tk()

button = tk.Button(root, text="Click Me")

button.pack()

root.mainloop()
```

4.checkbutton: The Checkbutton widget is used to display a number of options to a user as toggle buttons. The user can then select one or more options by clicking the button.

Syntax: w=Checkbutton(master, option, ...)

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Checkbutton")

var = tk.IntVar()

checkbutton = tk.Checkbutton(root, text="Select me", variable=var)

checkbutton.pack()

label = tk.Label(root, textvariable=var)

label.pack()

root.mainloop()
```

5.entry: The Entry widget accepts single line text strings.

Syntax: e=Entry(master, option, ...)

Ex:

```
import tkinter as tk

def show_text():
    print(entry.get())

root = tk.Tk()

entry = tk.Entry(root)

entry.pack()

button = tk.Button(root, text="Submit", command=show_text)

button.pack()

root.mainloop()
```

6.listbox: The listbox widget displays a list of items. The user can select the item from the given list. List Box can display different types of items.

Syntax: L=Listbox(master, option,...)

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Listbox")
```

```
listbox = tk.Listbox(root)
listbox.pack()
listbox.insert(1, "Apple")
listbox.insert(2, "Banana")
listbox.insert(3, "Cherry")
root.mainloop()
```

7.message: It is GUI element of tkinter. It is multiline and non-editable object. It displays the static text. If length of message is large, it automatically breaks the long text to multiple lines. It is similar to label widget.

Syntax: M=Message(master, option)

Ex:

```
import tkinter as tk
from tkinter import messagebox
root = tk.Tk()
root.withdraw()
messagebox.showinfo("Message", "Hello, this is a simple message!")
root.mainloop()
```

8.radiobutton: The radio button is also known as option button. The option button allows user to select values from the predefined set of values. Radio button contains text as well as image.

Syntax:

Ex:

```
import tkinter as tk
root = tk.Tk()
root.title("Radio Button Example")
selected_option = tk.StringVar(value="1")
tk.Radiobutton(root, text="PUBG", variable=selected_option, value="1").pack()
tk.Radiobutton(root, text="Free Fire", variable=selected_option, value="2").pack()
root.mainloop()
```

9.text: Text widget allows user to edit the multiline text. User can also format the text the way user want to it display.

Syntax: T=Text(master, option,...)

Ex:

```
import tkinter as tk

root = tk.Tk()

text_box = tk.Text(root)

text_box.pack()

root.mainloop()
```

10.spinbox: The spinbox contains the fixed number of values and it allows selecting the value from the given values. It is a standard Tkinter Entry widget.

Syntax: s=Spinbox(master, option)

Ex:

```
import tkinter as tk

root = tk.Tk()

root.title("Simple Spinbox Example")

spinbox = tk.Spinbox(root, from_=1, to=10)

spinbox.pack(pady=20)

root.mainloop()
```

Regular Expressions: Regular Expression, or a RegEx, is a sequence of characters that forms a search pattern. RegEx can be used to check if a string contains the specified search pattern.

Reg Ex Module: Python has a built-in package called re, which can be used to work with Regular Expressions.

Ex:

```
import re

txt="The rain in Spain"

x=re.search("^The.*Spain$",txt)

if x:

    print("Yes! There is a match!")

else:

    print("No match!")
```

RegEx Function:

Function	Description
Findall	Returns a list containing all matches.
Search	Returns a Match object if there is a match anywhere in the string.
Split	Returns a list where the string has been split at each match.
Sub	Replaces one or many matches with a string.

MetaCharacters:

Meta characters are characters with a special meaning.

Character	Description	Example
[]	A set of characters	"[a-m]"
\	Signals a special sequence (can also be used to escape special characters)	"\d"
.	Any character (except newline character)	"he..o"
^	Starts with	"^hello"
\$	Ends with	"world\$"
*	Zero or more occurrences	"aix*"
+	One or more occurrences	"aix+"
{}	Exactly the specified number of occurrences	"al{2}"
	Either or	"falls stays"
()	Capture and group	

1.[] Square Brackets: Square Brackets ([]) represents a character class consisting of a set of characters that we wish to match.

For example, the character class [abc] will match any single a, b, or c.

We can also specify a range of characters using “– “ inside the square brackets.

2.Dot (.): Dot(.) symbol matches only a single character except for the newline character (\n).

3.^ Caret: Caret(^) symbol matches the beginning of the string i.e. checks whether the string starts with the given character(s) or not.

For example: ^g will check if the string starts with g such as globe, girl, g, etc.

4.\$ Dollar: Dollar(\$) symbol matches the end of the string i.e checks whether the string ends with the given character(s) or not.

For example : s\$ will check for the string that ends with geeks, ends, s, etc.

5.| Or: Or symbol works as the or operator meaning it checks whether the pattern before or after the or symbol is present in the string or not.

For example: a|b will match any string that contains a or b such as acd, bcd, abcd, etc.

6.? Question Mark: Question mark(?) checks if the string before the question mark in the regex occurs at least once or not at all.

7.Star: Star (*) symbol matches zero or more occurrences of the regex preceding the * symbol.

8.+ Plus: Plus (+) symbol matches one or more occurrences of the regex preceding the + symbol.

9.{m, n} Braces: Braces matches any repetitions preceding regex from m to n both inclusive.

Special Sequences:

Character	Description	Example
<code>\A</code>	Returns a match if the specified characters are at the beginning of the string.	<code>"\AThe"</code>
<code>\b</code>	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\bain"</code> <code>r"ain\b"</code>
<code>\B</code>	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	<code>r"\Bain"</code> <code>r"ain\B"</code>
<code>\d</code>	Returns a match where the string contains digits (numbers from 0-9)	<code>"\d"</code>
<code>\D</code>	Returns a match where the string DOES NOT contain digits	<code>"\D"</code>
<code>\s</code>	Returns a match where the string contains a white space character	<code>"\s"</code>
<code>\S</code>	Returns a match where the string DOES NOT contain a white space character	<code>"\S"</code>

<code>\w</code>	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore <code>_</code> character)	<code>"\w"</code>
<code>\W</code>	Returns a match where the string DOES NOT contain any word characters	<code>"\W"</code>
<code>\Z</code>	Returns a match if the specified characters are at the end of the string	<code>"Spain\Z"</code>

1.findall(): The `findall()` function returns a list containing all matches.

Ex:

```
import re
text="The Rain in Spain"
x=re.findall("ai",text)
print(x)
```

NOTE: If no matches are found, an empty list is returned

2.search(): The `search()` function searches the string for a match, and returns a Match object if there is a match.

NOTE: If there is more than one match, only the first occurrence of the match will be returned.

If no matches are found, the value `None` is returned.

Ex:

```
import re
text="The Rain in Spain"
x=re.search("\s",text)
print("The first white-space position is at",x.start())
```

3.split(): The `split()` function returns a list where the string has been split at each match.

Ex:

```
import re
text="The Rain in Spain"
x=re.split("\s",text)
print(x)
```


Ex: Split the string only at the first occurrence.

```
import re
text="The Rain in Spain"
x=re.split("\s",text,2)
print(x)
```

4.sub(): The sub() function replaces the matches with the text of your choice.

Ex:

```
import re
text="The Rain in Spain"
x=re.sub("\s","_",text)
print(x)
```

NOTE: You can control the number of replacements by specifying the count parameter.

5.Match Object: A Match Object is an object containing information about the search and the result. Note: If there is no match, the value None will be returned, instead of the Match Object.

Ex:

```
import re
text="The Rain in Spain"
x=re.search("ai",text)
print(x)
```